```
import os
import re
import json
import logging
from datetime import datetime, date, time, timedelta
from statistics import median
from zoneinfo import ZoneInfo
from typing import Any, Dict, List, Optional, Tuple

import requests
from bs4 import BeautifulSoup
import anthropic
import gspread
from openai import OpenAI
from google.oauth2.service_account import Credentials
from telegram import Update
from telegram.ext import (
    Application,
    MessageHandler,
    CommandHandler,
    filters,
    ContextTypes,
)
from gspread_formatting import format_cell_range, CellFormat, Color, TextFormat

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

KYIV_TZ = ZoneInfo("Europe/Kyiv")
MINFIN_URL = "https://minfin.com.ua/currency/auction/usd/buy/dnepropetrovsk/"

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN", "")
CLAUDE_API_KEY = os.environ.get("CLAUDE_API_KEY")
OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")
SPREADSHEET_ID = os.environ.get("SPREADSHEET_ID", "")
GOOGLE_CREDS = os.environ.get("GOOGLE_CREDS", "")

ALLOWED_USERS_STR = os.environ.get("ALLOWED_USERS", "")
ALLOWED_USERS = [int(x.strip()) for x in ALLOWED_USERS_STR.split(",") if x.strip(

FULL_PLATES = [
    "AI1457MM", "AE0418OP", "AE2993PI", "AE7935PI", "KA3021EO", "KA9489EP",
    "AE7121TA", "AE8204TB", "AE2548TB", "AE9245TO", "AE0736PK", "AE4715TH",
```

```
    "AE6514TC", "KA4895HE", "KA6843HB", "AE5308TE", "BI1875HO", "KA0665IH",
    "KA0349HO", "BC9854PM", "AE8391TM", "AE4553XB", "KA8730IX", "AE5725OO",
    "CA6584KA", "AI3531PH",
]

TO_BUNDLE = [
    {"description": "Масло в двигатель", "amount": 780},
    {"description": "Воздушный фильтр WX WA9545", "amount": 270},
    {"description": "Газовые фильтра", "amount": 100},
    {"description": "Масляный фильтр BO 0451103318", "amount": 160},
    {"description": "Работы за ТО", "amount": 300},
]

SKIP_GRM = {"9245", "5308", "4715", "8204", "0736"}

INSURANCE_DATE_COL = 18  # R (1-based)
INSURANCE_COMPANY_COL = 19  # S (1-based)

REPORT_CACHE: Dict[str, Any] = {"snapshot": None, "time": None}
REPORT_CACHE_TTL = 600  # 10 хвилин — зменшує навантаження на Google Sheets API



def parse_insurance_a4(text) -> tuple:
    """Парсить рядок з A4: 'Страховка до 24.11.26 Євроінс' -> (date, company)"""
    if not text:
        return None, None
    s = str(text).strip()
    m = re.search(r'(\d{2}\.\d{2}\.\d{2,4})', s)
    if not m:
        return None, None
    date_str = m.group(1)
    try:
        fmt = '%d.%m.%y' if len(date_str) == 8 else '%d.%m.%Y'
        d   = datetime.strptime(date_str, fmt).date()
    except Exception:
        return None, None
    company = s[m.end():].strip() or 'Страховка'
    return d, company


def extract_digits(value: str) -> str:
    return "".join(re.findall(r"\d+", str(value or "")))


VEHICLE_MAP = {extract_digits(p): p for p in FULL_PLATES if extract_digits(p)}
KNOWN_CAR_IDS = sorted(VEHICLE_MAP.keys())
```

```
claude_client = anthropic.Anthropic(api_key=CLAUDE_API_KEY) if CLAUDE_API_KEY els
openai_client = OpenAI(api_key=OPENAI_API_KEY) if OPENAI_API_KEY else None


# ===== Formatting helpers =====

def blue_text_format() -> CellFormat:
    return CellFormat(textFormat=TextFormat(foregroundColor=Color(0, 0, 1)))


def yellow_fill_format() -> CellFormat:
    return CellFormat(backgroundColor=Color(1, 0.96, 0.75))


def apply_blue_text(ws, cell_range: str) -> None:
    try:
        format_cell_range(ws, cell_range, blue_text_format())
    except Exception as e:
        logger.error(f"Blue text format error: {e}")


def mark_cell_yellow(ws, cell_range: str) -> None:
    try:
        format_cell_range(ws, cell_range, yellow_fill_format())
    except Exception as e:
        logger.error(f"Yellow fill format error: {e}")


# ===== Google Sheets =====

_gspread_client = None
_gspread_client_ts = None


def get_sheet():
    global _gspread_client, _gspread_client_ts
    import time as _time
    now = datetime.now(KYIV_TZ)
    # Кешуємо gspread клієнт на 30 хвилин
    if _gspread_client is None or _gspread_client_ts is None or             (now
        creds_dict = json.loads(GOOGLE_CREDS)
        scopes = [
            "https://spreadsheets.google.com/feeds",
            "https://www.googleapis.com/auth/drive",
        ]
        creds = Credentials.from_service_account_info(creds_dict, scopes=scopes)
```

```
        _gspread_client    = gspread.authorize(creds)
        _gspread_client_ts = now

    # Retry з паузою при 429
    for attempt in range(3):
        try:
            return _gspread_client.open_by_key(SPREADSHEET_ID)
        except gspread.exceptions.APIError as e:
            if '429' in str(e) and attempt < 2:
                wait = 10 * (attempt + 1)
                logger.warning("Google Sheets 429, чекаємо %ds (спроба %d)", wait
                _time.sleep(wait)
            else:
                raise


_DRIVERS_CACHE: Dict[str, Dict] = {}
_DRIVERS_CACHE_TS: Optional[datetime] = None


def _load_drivers_cache() -> None:
    global _DRIVERS_CACHE, _DRIVERS_CACHE_TS
    now = datetime.now(KYIV_TZ)
    if _DRIVERS_CACHE_TS and (now - _DRIVERS_CACHE_TS).total_seconds() < 300:  #
        return
    try:
        creds_dict = json.loads(GOOGLE_CREDS)
        scopes     = ["https://spreadsheets.google.com/feeds",
                      "https://www.googleapis.com/auth/drive"]
        creds      = Credentials.from_service_account_info(creds_dict, scopes=sco
        client = gspread.authorize(creds)
        # Retry при 429
        import time as _time
        for _attempt in range(3):
            try:
                sp = client.open_by_key(DRIVERS_SPREADSHEET_ID)
                break
            except gspread.exceptions.APIError as _e:
                if '429' in str(_e) and _attempt < 2:
                    _time.sleep(10 * (_attempt + 1))
                else:
                    raise
        ws         = None
        for sheet in sp.worksheets():
            if "ТО" in sheet.title or "грм" in sheet.title.lower():
                ws = sheet
                break
```

```
        if not ws:
            ws = sp.sheet1
        cache = {}
        all_rows = ws.get_all_values()
        logger.info("Drivers sheet: %d rows, first row: %s", len(all_rows), all_r
        for row in all_rows[2:]:  # пропускаємо 2 рядки заголовків
            if not row or not str(row[0]).strip():
                continue
            key    = re.sub(r"[^0-9]", "", str(row[0]).strip())
            name   = str(row[11]).strip() if len(row) > 11 else ""
            phone1 = str(row[12]).strip() if len(row) > 12 else ""
            phone2 = str(row[13]).strip() if len(row) > 13 else ""
            if key:
                cache[key] = {"name": name, "phone1": phone1, "phone2": phone2}
                if name or phone1:
                    logger.debug("Driver: key=%s name=%s phone=%s", key, name, ph
        logger.info("Drivers cache loaded: %d entries", len(cache))
        _DRIVERS_CACHE    = cache
        _DRIVERS_CACHE_TS = now
    except Exception as e:
        logger.error("_load_drivers_cache: %s", e)


def fmt_driver(car_id: str) -> str:
    _load_drivers_cache()
    info   = _DRIVERS_CACHE.get(car_id, {})
    name   = info.get("name", "").strip()
    phone1 = info.get("phone1", "").strip()
    phone2 = info.get("phone2", "").strip()
    if not name and not phone1:
        return "Немає водія"
    phones = " / ".join(p for p in [phone1, phone2] if p)
    parts  = []
    if name:   parts.append(name)
    if phones: parts.append(phones)
    return " | ".join(parts)


def get_matching_worksheet(spreadsheet, car_id: str):
    full_plate = VEHICLE_MAP.get(car_id, "")
    for ws in spreadsheet.worksheets():
        title = str(ws.title)
        if car_id in title or (full_plate and full_plate in title):
            return ws
    return None
```

```python
def get_data_snapshot(force_refresh: bool = False) -> Dict[str, List[List[str]]]:
    global REPORT_CACHE
    now = datetime.now(KYIV_TZ)
    if not force_refresh and REPORT_CACHE["snapshot"] and REPORT_CACHE["time"]:
        if (now - REPORT_CACHE["time"]).total_seconds() < REPORT_CACHE_TTL:
            return REPORT_CACHE["snapshot"]

    spreadsheet = get_sheet()
    snapshot: Dict[str, List[List[str]]] = {}
    for ws in spreadsheet.worksheets():
        snapshot[ws.title] = ws.get_all_values()

    REPORT_CACHE = {"snapshot": snapshot, "time": now}
    return snapshot


# ===== Basic parsers =====

def parse_num(v) -> Optional[int]:
    if v is None:
        return None
    s = str(v).strip()
    digits = re.sub(r"[^\d\-]", "", s)
    if not digits:
        return None
    try:
        return int(digits)
    except ValueError:
        return None


def normalize_date_short(date_str: Optional[str]) -> str:
    if not date_str:
        return datetime.now(KYIV_TZ).strftime("%d.%m.%y")
    s = str(date_str).strip()
    for fmt in ("%d.%m.%Y", "%d.%m.%y", "%d-%m-%Y", "%d-%m-%y"):
        try:
            return datetime.strptime(s, fmt).strftime("%d.%m.%y")
        except ValueError:
            pass
    return datetime.now(KYIV_TZ).strftime("%d.%m.%y")


def parse_short_date(date_str) -> Optional[date]:
    if not date_str:
        return None
    # Handle datetime/date objects from openpyxl
```

```
    if hasattr(date_str, 'date'):
        return date_str.date()
    if isinstance(date_str, date):
        return date_str
    s = str(date_str).strip()
    if not s or s in ('None', ''):
        return None
    # Try ISO format first (from openpyxl string conversion)
    if 'T' in s or (len(s) > 8 and '-' in s[:8]):
        try:
            return datetime.fromisoformat(s.split(' ')[0].split('T')[0]).date()
        except Exception:
            pass
    for fmt in ("%d.%m.%Y", "%d.%m.%y", "%d-%m-%Y", "%d-%m-%y",
                "%Y-%m-%d", "%Y.%m.%d"):
        try:
            return datetime.strptime(s, fmt).date()
        except ValueError:
            pass
    return None


def format_km(v: Optional[int]) -> str:
    if v is None:
        return ""
    sign = "-" if v < 0 else ""
    return f"{sign}{abs(v):,}".replace(",", ".")


def resolve_car_id(value: Optional[str]) -> Optional[str]:
    if not value:
        return None
    raw = str(value).strip().upper()
    digits = extract_digits(raw)
    if digits in VEHICLE_MAP:
        return digits
    for short_id, full_plate in VEHICLE_MAP.items():
        if raw == full_plate.upper():
            return short_id
    return None


def full_plate_from_short(car_id: Optional[str]) -> str:
    if not car_id:
        return "Невідомо"
    return VEHICLE_MAP.get(car_id, car_id)
```

```
def clean_json_text(text: str) -> str:
    if not text:
        return ""
    s = text.strip().replace("```json", "").replace("```", "").strip()
    start = s.find("{")
    end = s.rfind("}")
    if start != -1 and end != -1 and end > start:
        return s[start:end + 1]
    return s


# ===== AI parsing =====

def build_known_cars_block() -> str:
    return "\n".join(f"{k} -> {VEHICLE_MAP[k]}" for k in KNOWN_CAR_IDS)


def build_prompt(message: str, existing_data: Optional[dict] = None) -> str:
    today = datetime.now(KYIV_TZ).strftime("%d.%m.%y")
    existing_block = ""
    if existing_data:
        existing_block = f'\nAlready known data:\n{json.dumps(existing_data, ensu
    cars_block = build_known_cars_block()
    return f"""Ты помощник для учета автопарка. Сегодня {today}.

Твоя задача: разобрать сообщение пользователя в СТРОГИЙ JSON для записи в Google
{existing_block}
Известные машины автопарка:
{cars_block}

Правила:
1. Пользователь может писать данные в любом порядке.
2. Пользователь часто пишет только цифры машины, например 4553 или 8730.
3. car_id в JSON должен быть только из списка известных машин.
4. Если дата не указана - используй сегодняшнюю дату в формате DD.MM.YY.
5. ДАННЫЕ ДЛЯ ТАБЛИЦЫ ПИШИ НА РУССКОМ ЯЗЫКЕ.
6. Ответ должен быть ТОЛЬКО JSON.
7. Если не хватает важных данных - верни missing_fields.
8. Если пользователь пишет "ТО" или "плановое ТО", description верни как "ТО".
9. Если пользователь пишет про штраф, долг, должен, дожен - type верни как "liabi
10. Если пользователь пишет "взял", "принял", "погасил", "дал" в контексте долга
11. Для liability_minus и liability_plus odometer не нужен.
12. Для liability_minus и liability_plus description должна быть только текстом п
13. Для income description может быть пустым.

Сообщение пользователя:
```

```
"{message}"

Верни JSON строго такого вида:
{{
  "type": "expense" или "income" или "liability_minus" или "liability_plus" или n
  "car_id": "8730" или null,
  "date": "DD.MM.YY",
  "amount": 370,
  "description": "Колодки Бош",
  "odometer": 470420,
  "notes": null,
  "missing_fields": []
}}"""


def ask_claude(prompt: str) -> dict:
    if not claude_client:
        raise Exception("CLAUDE_API_KEY not set")
    response = claude_client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=700,
        messages=[{"role": "user", "content": prompt}],
    )
    text = response.content[0].text.strip()
    return json.loads(clean_json_text(text))


def ask_openai(prompt: str) -> dict:
    if not openai_client:
        raise Exception("OPENAI_API_KEY not set")
    response = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": "Возвращай только валидный JSON. Без по
            {"role": "user", "content": prompt},
        ],
    )
    text = response.choices[0].message.content.strip()
    return json.loads(clean_json_text(text))


def ask_ai(message: str, existing_data: Optional[dict] = None) -> dict:
    prompt = build_prompt(message, existing_data)
    if claude_client:
        try:
            return ask_claude(prompt)
```

```
        except Exception as e:
            logger.error(f"Claude error: {e}")
    if openai_client:
        try:
            return ask_openai(prompt)
        except Exception as e:
            logger.error(f"OpenAI error: {e}")
            return {"error": f"AI недоступний: {e}"}
    return {"error": "Не задані CLAUDE_API_KEY і OPENAI_API_KEY"}


# ===== Special parsing =====

def is_to_phrase(text: str) -> bool:
    t = str(text or "").lower().strip()
    return t == "то" or " то " in f" {t} " or "плановое то" in t or "планове то"


def detect_liability_type(text: str) -> Optional[str]:
    t = str(text or "").lower().strip()
    plus_markers = ["взял", "принял", "погасил", "дал "]
    minus_markers = ["штраф", "долг", "должен", "должна", "дожен"]
    if any(marker in t for marker in plus_markers):
        return "liability_plus"
    if any(marker in t for marker in minus_markers):
        return "liability_minus"
    return None


def apply_special_cases(data: dict, raw_text: str) -> dict:
    liability_type = detect_liability_type(raw_text)
    if liability_type and not data.get("type"):
        data["type"] = liability_type
    if is_to_phrase(raw_text):
        data.setdefault("type", "expense")
        data.setdefault("description", "ТО")
        if data.get("amount") in ("", None):
            data["amount"] = 0
    return data


def compute_missing_fields(data: dict, raw_text: str = "") -> list[str]:
    missing = []
    op_type = data.get("type")
    to_case = is_to_phrase(raw_text) or str(data.get("description", "")).lower().

    if not op_type:
```

```
        missing.append("type")
    if not data.get("car_id"):
        missing.append("car_id")
    if data.get("amount") in (None, "") and not to_case:
        missing.append("amount")
    if op_type in ["expense", "liability_minus", "liability_plus"] and not data.g
        missing.append("description")
    if op_type in ["expense", "income"] and data.get("odometer") in (None, ""):
        missing.append("odometer")
    return missing


def ask_for_next_missing_field(missing_fields: list[str]) -> str:
    if not missing_fields:
        return "Уточни, будь ласка, відсутні дані."
    field = missing_fields[0]
    mapping = {
        "type": "Вкажи, будь ласка, це прихід, витрата, штраф чи борг.",
        "car_id": f"Вкажи номер машини. Доступні: {', '.join(KNOWN_CAR_IDS)}",
        "amount": "Вкажи суму в гривнях.",
        "description": "Вкажи опис або причину.",
        "odometer": "Мені додати середньостатистичний пробіг? Напиши «так» або пр
    }
    return mapping.get(field, "Уточни, будь ласка, відсутні дані.")


def build_liability_description(op_type: str, raw_text: str, ai_description: Opti
    t = str(raw_text or "").lower()
    desc = str(ai_description or "").strip()
    if "дтп" in t:
        base = "за ДТП"
    elif "телевиз" in t:
        base = "за телевизор"
    elif "парков" in t:
        base = "за парковку"
    elif "превыш" in t:
        base = "за превышение"
    elif "штраф" in t and op_type == "liability_plus":
        base = "за штраф"
    elif desc:
        base = desc if desc.lower().startswith("за ") else f"за {desc}"
    else:
        base = ""
    if op_type == "liability_minus":
        return f"{'штраф' if 'штраф' in t else 'долг'} {base}".strip()
    return f"погашение долга {base}".strip()
```

```
def heuristic_multi_parse(text: str) -> Optional[List[dict]]:
    t = str(text or "").strip()
    if "," in t:
        actions: List[dict] = []
        shared_car_id = None
        for car_id in KNOWN_CAR_IDS:
            if re.search(rf"(?<!\d){re.escape(car_id)}(?!\d)", t):
                shared_car_id = car_id
                break
        if not shared_car_id:
            return None
        parts = [p.strip() for p in t.split(",") if p.strip()]
        for part in parts:
            low = part.lower()
            nums = [int(x) for x in re.findall(r"\d+", part)]
            amounts = [n for n in nums if str(n) != shared_car_id and str(n) not
            if "приход" in low and amounts:
                actions.append({
                    "type": "income", "car_id": shared_car_id, "date": normalize_
                    "amount": max(amounts), "description": "", "odometer": None,
                })
            elif detect_liability_type(low) == "liability_minus" and amounts:
                actions.append({
                    "type": "liability_minus", "car_id": shared_car_id, "date": n
                    "amount": amounts[0], "description": build_liability_descript
                    "odometer": None, "notes": None, "missing_fields": []
                })
            elif detect_liability_type(low) == "liability_plus" and amounts:
                actions.append({
                    "type": "liability_plus", "car_id": shared_car_id, "date": no
                    "amount": amounts[0], "description": build_liability_descript
                    "odometer": None, "notes": None, "missing_fields": []
                })
        return actions or None

    shared_car_id = None
    for car_id in KNOWN_CAR_IDS:
        if re.search(rf"(?<!\d){re.escape(car_id)}(?!\d)", t):
            shared_car_id = car_id
            break
    if not shared_car_id:
        return None

    nums = [int(x) for x in re.findall(r"\d+", t)]
    amounts = [n for n in nums if str(n) != shared_car_id and str(n) not in KNOWN
    liability_type = detect_liability_type(t)
```

```
    if is_to_phrase(t):
        return [{
            "type": "expense", "car_id": shared_car_id, "date": normalize_date_sh
            "amount": 0, "description": "TO", "odometer": None, "notes": None, "m
        }]
    if liability_type == "liability_minus" and amounts:
        return [{
            "type": "liability_minus", "car_id": shared_car_id, "date": normalize
            "amount": amounts[0], "description": build_liability_description("lia
            "odometer": None, "notes": None, "missing_fields": []
        }]
    if liability_type == "liability_plus" and len(amounts) == 1:
        return [{
            "type": "income", "car_id": shared_car_id, "date": normalize_date_sho
            "amount": amounts[0], "description": "", "odometer": None, "notes": N
        }]
    if liability_type == "liability_plus" and len(amounts) >= 2:
        sorted_amounts = sorted(amounts, reverse=True)
        actions = [{
            "type": "income", "car_id": shared_car_id, "date": normalize_date_sho
            "amount": sorted_amounts[0], "description": "", "odometer": None, "no
        }]
        for extra in sorted_amounts[1:]:
            actions.append({
                "type": "liability_plus", "car_id": shared_car_id, "date": normal
                "amount": extra, "description": build_liability_description("liab
                "odometer": None, "notes": None, "missing_fields": []
            })
        return actions
    return None


# ===== Reports: current odometer and service blocks =====

def get_current_odometer_from_rows(rows: List[List[str]]) -> Optional[int]:
    # Беремо одометр з останньої за ДАТОЮ записі.
    # Колонка F (витрати) та L (доходи) порівнюємо по даті — перемагає пізніша.
    latest_f: Optional[Tuple[date, int]] = None
    latest_l: Optional[Tuple[date, int]] = None

    for r in rows[7:]:
        if len(r) > 5:
            d   = parse_short_date(r[4] if len(r) > 4 else None)
            odo = parse_num(r[5])
            if d and odo is not None and odo > 1000:
                if latest_f is None or d > latest_f[0] or (d == latest_f[0] and o
```

```
                latest_f = (d, odo)
        if len(r) > 11:
            d   = parse_short_date(r[10] if len(r) > 10 else None)
            odo = parse_num(r[11])
            if d and odo is not None and odo > 1000:
                if latest_l is None or d > latest_l[0] or (d == latest_l[0] and o
                    latest_l = (d, odo)

    if latest_f and latest_l:
        if latest_f[0] > latest_l[0]:
            return latest_f[1]
        if latest_l[0] > latest_f[0]:
            return latest_l[1]
        return max(latest_f[1], latest_l[1])
    if latest_f:
        return latest_f[1]
    if latest_l:
        return latest_l[1]
    return None


def split_expense_blocks(rows: List[List[str]]) -> List[List[Dict[str, Any]]]:
    blocks: List[List[Dict[str, Any]]] = []
    current: List[Dict[str, Any]] = []
    current_date = ""
    current_odo = None

    for row in rows[7:]:
        e = row[4] if len(row) > 4 else ""
        f = parse_num(row[5] if len(row) > 5 else None)
        g = str(row[6]).strip() if len(row) > 6 else ""
        h = parse_num(row[7] if len(row) > 7 else None)
        i = row[8] if len(row) > 8 else ""

        new_block = False
        if e and f is not None:
            if current:
                new_block = True
            current_date = e
            current_odo = f

        if new_block:
            blocks.append(current)
            current = []

        if current_date and current_odo is not None and any([e, f is not None, g,
            current.append({
```

```
                "date": current_date,
                "odo": current_odo,
                "desc": g.lower(),
                "amount": h,
            })

    if current:
        blocks.append(current)
    return blocks


def score_oil_block(block: List[Dict[str, Any]]) -> int:
    text = " | ".join(x["desc"] for x in block)
    score = 0
    # Сильні маркери (однозначно заміна масла)
    if "масло в двигатель" in text:
        score += 10
    if "моторное масло" in text:
        score += 8
    if "замена масла" in text:
        score += 8
    if "заміна масла" in text:
        score += 8
    if "масло в двигун" in text:
        score += 8
    if "моторне масло" in text:
        score += 8
    # Слабкі маркери
    if "масляный фильтр" in text:
        score += 4
    if "масляний фільтр" in text:
        score += 4
    if "масло" in text:
        score += 2
    return score


def score_grm_block(block: List[Dict[str, Any]]) -> int:
    text = " | ".join(x["desc"] for x in block)
    score = 0
    if "комплект грм" in text:
        score += 10
    if "замена грм" in text or "замана грм" in text:
        score += 10
    if "ремень грм" in text:
        score += 7
    if "ролик грм" in text:
```

```
        score += 6
    if "грм" in text:
        score += 4
    if "помпа" in text:
        score += 2
    return score


def find_last_service(rows: List[List[str]], mode: str) -> Tuple[Optional[str], O
    blocks = split_expense_blocks(rows)
    if not blocks:
        return None, None

    scorer = score_oil_block if mode == "oil" else score_grm_block
    for block in reversed(blocks):
        if scorer(block) >= (8 if mode == "oil" else 8):
            return block[0]["date"], block[0]["odo"]
    return None, None


def get_color_icon(remaining: Optional[int], total: int) -> str:
    if remaining is None:
        return "⚪"
    if remaining <= 1000:
        return "🔴"
    ratio = remaining / total
    if ratio > 0.66:
        return "🟢"
    if ratio > 0.33:
        return "🟡"
    return "🟠"


def build_oil_report() -> str:
    snapshot = get_data_snapshot()
    items = []
    for car_id in KNOWN_CAR_IDS:
        rows = None
        for title, data in snapshot.items():
            if car_id in title or VEHICLE_MAP.get(car_id, "") in title:
                rows = data
                break
        if not rows:
            continue
        last_date, last_odo = find_last_service(rows, "oil")
        current_odo = get_current_odometer_from_rows(rows)
        if last_odo is None or current_odo is None:
```

```
            continue
        if current_odo < last_odo:
            current_odo = last_odo
        remaining = 10000 - (current_odo - last_odo)
        icon = get_color_icon(remaining, 10000)
        line = f"{icon} {car_id} | {last_date} | {last_odo} | {format_km(remainin
        if remaining <= 1000:
            drv = fmt_driver(car_id)
            line += f"\n   👤 {drv}"
        items.append((remaining, line))
    items.sort(key=lambda x: x[0])
    return "\n".join(line for _, line in items)


def build_grm_report() -> str:
    snapshot = get_data_snapshot()
    items = []
    for car_id in KNOWN_CAR_IDS:
        if car_id in SKIP_GRM:
            continue
        rows = None
        for title, data in snapshot.items():
            if car_id in title or VEHICLE_MAP.get(car_id, "") in title:
                rows = data
                break
        if not rows:
            continue
        last_date, last_odo = find_last_service(rows, "grm")
        current_odo = get_current_odometer_from_rows(rows)
        if last_odo is None or current_odo is None:
            continue
        if current_odo < last_odo:
            current_odo = last_odo
        remaining = 50000 - (current_odo - last_odo)
        icon = get_color_icon(remaining, 50000)
        line = f"{icon} {car_id} | {last_date} | {last_odo} | {format_km(remainin
        if remaining <= 1000:
            drv = fmt_driver(car_id)
            line += f"\n   👤 {drv}"
        items.append((remaining, line))
    items.sort(key=lambda x: x[0])
    return "\n".join(line for _, line in items)


# ===== Insurance =====

def insurance_days_icon(days_left: int) -> str:
```

```
    if days_left <= 14:
        return "🔴"
    if days_left <= 30:
        return "🟠"
    if days_left <= 90:
        return "🟡"
    return "🟢"


def build_insurance_report() -> str:
    snapshot = get_data_snapshot()
    today = datetime.now(KYIV_TZ).date()
    lines = []
    for car_id in KNOWN_CAR_IDS:
        rows = None
        for title, data in snapshot.items():
            if car_id in title or VEHICLE_MAP.get(car_id, "") in title:
                rows = data
                break
        if not rows:
            continue

        # Спочатку шукаємо в A4 (рядок 4, індекс 3)
        best: Optional[Tuple[date, str]] = None
        if len(rows) > 3 and rows[3] and rows[3][0]:
            d, company = parse_insurance_a4(rows[3][0])
            if d:
                best = (d, company)
        # Якщо в A4 немає — шукаємо в колонках R/S
        if not best:
            for row in rows[7:]:
                if len(row) >= INSURANCE_COMPANY_COL:
                    d = parse_short_date(row[INSURANCE_DATE_COL - 1])
                    company = str(row[INSURANCE_COMPANY_COL - 1]).strip()
                    if d and company:
                        if best is None or d > best[0]:
                            best = (d, company)
        if not best:
            continue
        end_date, company = best
        days_left = (end_date - today).days
        icon = insurance_days_icon(days_left)
        lines.append((days_left, f"{icon} {car_id} | {end_date.strftime('%d.%m.%y
    lines.sort(key=lambda x: x[0])
    return "\n".join(line for _, line in lines)
```

```
async def check_service_and_insurance_notifications(context: ContextTypes.DEFAULT
    now_kyiv = datetime.now(KYIV_TZ)
    if now_kyiv.weekday() >= 5:
        logger.info("Notify skipped: weekend")
        return
    logger.info("Running daily notification check...")
    try:
        await _run_notifications(context)
    except Exception as e:
        logger.error("Notify top-level error: %s", e, exc_info=True)
        for user_id in ALLOWED_USERS:
            try:
                await context.bot.send_message(
                    chat_id=user_id,
                    text=f"⚠️ Помилка при формуванні регламентів: {e}")
            except Exception:
                pass


async def _run_notifications(context: ContextTypes.DEFAULT_TYPE):
    now_kyiv = datetime.now(KYIV_TZ)
    snapshot = get_data_snapshot(force_refresh=True)
    today = now_kyiv.date()
    alert_items: List[Tuple[int, str]] = []

    for car_id in KNOWN_CAR_IDS:
        rows = None
        for title, data in snapshot.items():
            if car_id in title or VEHICLE_MAP.get(car_id, "") in title:
                rows = data
                break
        if not rows:
            continue

        current_odo = get_current_odometer_from_rows(rows)

        oil_date, oil_odo = find_last_service(rows, "oil")
        if oil_odo is not None and current_odo is not None:
            remaining = 10000 - (max(current_odo, oil_odo) - oil_odo)
            if remaining <= 1000:
                icon = "🔴" if remaining <= 0 else "🟠"
                drv = fmt_driver(car_id)
                drv_line = f"\n    👤 {drv}"
                if remaining < 0:
                    alert_items.append((remaining, f"{icon} {car_id} — масло прос
                else:
                    alert_items.append((remaining, f"{icon} {car_id} — масло чере
```

```
        if car_id not in SKIP_GRM:
            grm_date, grm_odo = find_last_service(rows, "grm")
            if grm_odo is not None and current_odo is not None:
                remaining = 50000 - (max(current_odo, grm_odo) - grm_odo)
                if remaining <= 1000:
                    icon = "🔴" if remaining <= 0 else "🟠"
                    drv = fmt_driver(car_id)
                    drv_line = f"\n    👤 {drv}"
                    if remaining < 0:
                        alert_items.append((remaining, f"{icon} {car_id} — ГРМ пр
                    else:
                        alert_items.append((remaining, f"{icon} {car_id} — ГРМ че

        # Спочатку шукаємо в A4
        best: Optional[Tuple[date, str]] = None
        if len(rows) > 3 and rows[3] and rows[3][0]:
            d, company = parse_insurance_a4(rows[3][0])
            if d:
                best = (d, company)
        if not best:
            for row in rows[7:]:
                if len(row) >= INSURANCE_COMPANY_COL:
                    d = parse_short_date(row[INSURANCE_DATE_COL - 1])
                    company = str(row[INSURANCE_COMPANY_COL - 1]).strip()
                    if d and company:
                        if best is None or d > best[0]:
                            best = (d, company)
        if best:
            end_date, company = best
            days_left = (end_date - today).days
            if days_left <= 14:
                if days_left < 0:
                    alert_items.append((days_left, f"🚗 {car_id} — страховка прос
                else:
                    alert_items.append((days_left, f"🚗 {car_id} — страховка чере

    logger.info("Notify: %d alert items found", len(alert_items))
    if alert_items:
        alert_items.sort(key=lambda x: x[0])
        text = "⚠️ Стан регламентів на сьогодні:\n\n" + "\n".join(msg for _, msg
        for user_id in ALLOWED_USERS:
            try:
                await context.bot.send_message(chat_id=user_id, text=text)
                logger.info("Notification sent to %s", user_id)
            except Exception as e:
                logger.error("Notification send error: %s", e)
```

```
    else:
        logger.info("Notify: no alerts today, message not sent")


# ===== USD rate =====

def get_usd_black_rate_dnipro() -> Optional[float]:
    headers = {"User-Agent": "Mozilla/5.0"}
    resp = requests.get(MINFIN_URL, headers=headers, timeout=15)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "html.parser")
    text = soup.get_text(" ", strip=True)
    patterns = [
        r"Средняя покупка\s*([0-9]+[.,][0-9]+)",
        r"Середня купівля\s*([0-9]+[.,][0-9]+)",
        r"Покупка\s*([0-9]+[.,][0-9]+)",
    ]
    for pattern in patterns:
        m = re.search(pattern, text, re.IGNORECASE)
        if m:
            return float(m.group(1).replace(",", "."))
    matches = re.findall(r"\b([0-9]{2}[.,][0-9]{2})\b", text)
    for val in matches:
        num = float(val.replace(",", "."))
        if 35 <= num <= 50:
            return num
    return None


# ===== Duplicate / anomaly =====

def get_last_8_weekly_points(ws) -> List[Tuple[date, int]]:
    all_vals = ws.get_all_values()
    points = []
    for row in all_vals[7:]:
        d = parse_short_date(row[10] if len(row) > 10 else None)
        odo = parse_num(row[11] if len(row) > 11 else None)
        if d and odo is not None:
            points.append((d, odo))
    return points[-8:]


def estimate_odometer_for_car(car_id: str, target_date_str: Optional[str] = None)
    spreadsheet = get_sheet()
    ws = get_matching_worksheet(spreadsheet, car_id)
    if not ws:
        return None
```

```
    points = get_last_8_weekly_points(ws)
    if not points:
        return None
    target_date = parse_short_date(target_date_str) or datetime.now(KYIV_TZ).date
    last_date, last_odo = points[-1]
    if target_date <= last_date:
        return last_odo

    daily_rates = []
    for i in range(1, len(points)):
        prev_date, prev_odo = points[i - 1]
        curr_date, curr_odo = points[i]
        delta_days = (curr_date - prev_date).days
        delta_km = curr_odo - prev_odo
        if delta_days > 0 and 0 <= delta_km <= 7000:
            rate = delta_km / delta_days
            if 0 <= rate <= 300:
                daily_rates.append(rate)
    if daily_rates:
        median_daily_rate = median(daily_rates)
        future_days = (target_date - last_date).days
        return int(round(last_odo + median_daily_rate * future_days))
    return last_odo


def odometer_is_anomalous(ws, new_odometer: int, operation_date_str: Optional[str
    points = get_last_8_weekly_points(ws)
    if not points:
        return False
    last_date, last_odo = points[-1]
    target_date = parse_short_date(operation_date_str) or datetime.now(KYIV_TZ).d
    if new_odometer <= last_odo:
        return False
    delta_km = new_odometer - last_odo
    delta_days = max((target_date - last_date).days, 1)
    weekly_equivalent = delta_km * 7 / delta_days
    return weekly_equivalent > 2500


def detect_duplicate(ws, action: dict, raw_text: str = "") -> bool:
    all_vals = ws.get_all_values()
    op_type = action.get("type")
    if op_type == "expense":
        for row in reversed(all_vals[7:]):
            if len(row) >= 9 and any(str(x).strip() for x in row[4:9]):
                last_date = str(row[4]).strip() if len(row) > 4 else ""
                last_odo = parse_num(row[5] if len(row) > 5 else None)
```

```
                last_desc = str(row[6]).strip().lower() if len(row) > 6 else ""
                last_amount = parse_num(row[7] if len(row) > 7 else None)
                return (
                    last_date == normalize_date_short(action.get("date"))
                    and last_odo == parse_num(action.get("odometer"))
                    and last_amount == parse_num(action.get("amount"))
                    and last_desc == str(action.get("description", "")).strip().l
                )
        return False
    if op_type == "income":
        for row in reversed(all_vals[7:]):
            if len(row) >= 15 and any(str(x).strip() for x in row[10:15]):
                last_date = str(row[10]).strip()
                last_odo = parse_num(row[11] if len(row) > 11 else None)
                last_amount = parse_num(row[12] if len(row) > 12 else None)
                return (
                    last_date == normalize_date_short(action.get("date"))
                    and last_odo == parse_num(action.get("odometer"))
                    and last_amount == parse_num(action.get("amount"))
                )
        return False
    if op_type in ["liability_minus", "liability_plus"]:
        current_desc = build_liability_description(op_type, raw_text, action.get(
        current_amount = -abs(float(action.get("amount", 0))) if op_type == "liab
        for row in reversed(all_vals[7:]):
            if len(row) >= 17 and any(str(x).strip() for x in row[15:17]):
                last_date = str(row[10]).strip() if len(row) > 10 else ""
                last_amount = str(row[15]).strip() if len(row) > 15 else ""
                last_desc = str(row[16]).strip().lower() if len(row) > 16 else ""
                amount_str = str(int(current_amount)) if float(current_amount).is
                return (
                    last_date == normalize_date_short(action.get("date"))
                    and last_amount == amount_str
                    and last_desc == current_desc
                )
        return False
    return False


# ===== Write to sheet =====

def get_last_used_row_for_block(ws, start_col: int, end_col: int, start_row: int
    all_vals = ws.get_all_values()
    last_used = start_row - 1
    for row_idx in range(start_row, len(all_vals) + 1):
        row = all_vals[row_idx - 1]
        block = row[start_col - 1:end_col]
```

```
        if any(str(cell).strip() for cell in block):
            last_used = row_idx
    return last_used


def get_next_expense_row(ws) -> int:
    return get_last_used_row_for_block(ws, 5, 9, 8) + 1


def get_next_right_block_row(ws) -> int:
    return max(get_last_used_row_for_block(ws, 11, 15, 8), get_last_used_row_for_


def get_previous_income_odometer(ws) -> Optional[int]:
    all_vals = ws.get_all_values()
    odometers = []
    for row in all_vals[7:]:
        if len(row) > 11:
            value = parse_num(row[11])
            if value is not None:
                odometers.append(value)
    return odometers[-1] if odometers else None


def write_expense_rows(ws, date_value: str, odometer: int, items: List[Dict[str,
    start_row = get_next_expense_row(ws)
    rows = []
    for item in items:
        amount = float(item["amount"])
        usd_amount = round(amount / usd_rate, 2) if usd_rate else ""
        rows.append([date_value, odometer, item["description"], amount, usd_amoun
    end_row = start_row + len(rows) - 1
    rng = f"E{start_row}:I{end_row}"
    ws.update(rng, rows)
    apply_blue_text(ws, rng)
    if odometer_estimated:
        for row_idx in range(start_row, end_row + 1):
            mark_cell_yellow(ws, f"F{row_idx}")
    return start_row, end_row, sum(float(x["amount"]) for x in items)


def write_single_action_to_sheet(data: dict, raw_text: str = "") -> str:
    spreadsheet = get_sheet()
    car_id = str(data.get("car_id", "")).strip()
    full_plate = full_plate_from_short(car_id)

    date_value = normalize_date_short(data.get("date"))
```

```
amount = float(data.get("amount", 0) or 0)
odometer = data.get("odometer", "")
description = data.get("description", "")
odometer_estimated = bool(data.get("odometer_estimated", False))
op_type = data.get("type")

usd_rate = None
usd_note = ""
try:
    usd_rate = get_usd_black_rate_dnipro()
    if usd_rate:
        usd_note = f"\n💱 Курс USD: {usd_rate}"
except Exception as e:
    logger.error(f"USD rate error: {e}")
    usd_note = "\n⚠️ Курс USD не вдалося отримати"

ws = get_matching_worksheet(spreadsheet, car_id)
if not ws:
    return f"❌ Машину {full_plate} не знайдено в таблиці"
sheet_name = ws.title

if op_type == "expense":
    desc_lower = str(description).lower().strip()
    if desc_lower in {"то", "плановое то", "планове то"} or is_to_phrase(desc
        start_row, end_row, total_amount = write_expense_rows(ws, date_value,
        return (
            f"✅ ТО внесено!\n🚗 Машина: {full_plate}\n📃 Додано 5 рядків\n"
            f"💸 Загальна сума: {total_amount} грн\n📅 {date_value}\n"
            f"📍 Внесено: лист '{sheet_name}', рядки {start_row}-{end_row}, с
        )

    next_row = get_next_expense_row(ws)
    usd_amount = round(amount / usd_rate, 2) if usd_rate else ""
    rng = f"E{next_row}:I{next_row}"
    ws.update(rng, [[date_value, odometer, description, amount, usd_amount]])
    apply_blue_text(ws, rng)
    if odometer_estimated:
        mark_cell_yellow(ws, f"F{next_row}")
    return (
        f"✅ Витрата внесена!\n🚗 Машина: {full_plate}\n📋 {description}\n💸
        f"📅 {date_value}\n📍 Внесено: лист '{sheet_name}', рядок {next_row},
    )

if op_type == "income":
    next_row = get_next_right_block_row(ws)
    usd_amount = round(amount / usd_rate, 2) if usd_rate else ""
    prev_odo = get_previous_income_odometer(ws)
```

```
        mileage_delta = ""
        if prev_odo is not None and odometer not in ("", None):
            try:
                mileage_delta = int(odometer) - int(prev_odo)
            except Exception:
                mileage_delta = ""
        rng = f"K{next_row}:O{next_row}"
        ws.update(rng, [[date_value, odometer, amount, usd_amount, mileage_delta]
        apply_blue_text(ws, rng)
        if odometer_estimated:
            mark_cell_yellow(ws, f"L{next_row}")
        delta_text = f"\n📈 Різниця пробігу: {mileage_delta}" if mileage_delta !=
        return (
            f"✅ Дохід внесено!\n🚘 Машина: {full_plate}\n💰 {amount} грн\n📅 {d
            f"📍 Внесено: лист '{sheet_name}', рядок {next_row}, стовпці K:O{delt
        )

    if op_type in ["liability_minus", "liability_plus"]:
        next_row = get_next_right_block_row(ws)
        sign_amount = -abs(amount) if op_type == "liability_minus" else abs(amoun
        liability_desc = build_liability_description(op_type, raw_text, descripti
        rng = f"K{next_row}:Q{next_row}"
        ws.update(rng, [[date_value, "", "", "", "", sign_amount, liability_desc]
        apply_blue_text(ws, rng)
        label = "Штраф/борг" if op_type == "liability_minus" else "Погашення/надх
        return (
            f"✅ {label} внесено!\n🚘 Машина: {full_plate}\n💱 {sign_amount} грн
            f"📍 Внесено: лист '{sheet_name}', рядок {next_row}, стовпці P:Q"
        )

    return "❌ Невідомий тип операції"


def write_actions_to_sheet(actions: List[dict], raw_text: str = "") -> str:
    return "\n\n".join(write_single_action_to_sheet(action, raw_text=raw_text) fo


def is_yes_statistical(text: str) -> bool:
    return str(text).lower().strip() in {"так", "да", "yes", "ок", "окей", "ага"}


def is_yes_confirm(text: str) -> bool:
    return str(text).lower().strip() in {"так", "да", "yes", "новий", "новая", "н


def is_no_confirm(text: str) -> bool:
    return str(text).lower().strip() in {"ні", "нет", "дубль", "скасувати", "отме
```

```
def actions_need_odometer(actions: List[dict]) -> bool:
    return any(action.get("type") in ["expense", "income"] and action.get("odomet


def fill_odometer_for_actions(actions: List[dict], odometer_value: int, estimated
    for action in actions:
        if action.get("type") in ["expense", "income"] and action.get("odometer")
            action["odometer"] = odometer_value
            action["odometer_estimated"] = estimated


def detect_month_summary_request(text: str) -> Optional[str]:
    t = str(text or "").lower()
    if any(x in t for x in ["місяць", "месяц", "поточний місяць", "текущий месяц"
        for car_id in KNOWN_CAR_IDS:
            if re.search(rf"(?<!\d){re.escape(car_id)}(?!\d)", text):
                return car_id
    return None


def monthly_summary(car_id: str) -> str:
    spreadsheet = get_sheet()
    ws = get_matching_worksheet(spreadsheet, car_id)
    if not ws:
        return f"❌ Машину {car_id} не знайдено в таблиці"

    today = datetime.now(KYIV_TZ)
    month = today.month
    year = today.year

    all_vals = ws.get_all_values()
    income_sum = 0.0
    expense_sum = 0.0
    liability_sum = 0.0

    for row in all_vals[7:]:
        if len(row) > 7:
            d = parse_short_date(row[4] if len(row) > 4 else None)
            num = parse_num(row[7] if len(row) > 7 else None)
            if d and d.month == month and d.year == year and num is not None:
                expense_sum += num
        if len(row) > 12:
            d = parse_short_date(row[10] if len(row) > 10 else None)
            num = parse_num(row[12] if len(row) > 12 else None)
            if d and d.month == month and d.year == year and num is not None:
```

```
                income_sum += num
        if len(row) > 15:
            d = parse_short_date(row[10] if len(row) > 10 else None)
            raw_p = row[15] if len(row) > 15 else None
            if d and d.month == month and d.year == year and str(raw_p).strip():
                try:
                    liability_sum += float(str(raw_p).replace(",", "."))
                except ValueError:
                    pass

    def fmt(x: float) -> str:
        return str(int(x)) if x.is_integer() else str(round(x, 2))
    return (
        f"📊 За поточний місяць по {car_id}:\n"
        f"💰 Дохід: {fmt(income_sum)} грн\n"
        f"💸 Витрати: {fmt(expense_sum)} грн\n"
        f"📌 Залишок боргу: {fmt(liability_sum)} грн"
    )


# ===== Command detectors =====

def is_oil_report_request(text: str) -> bool:
    t = str(text or "").lower().strip()
    return t in {"масло", "замена масла", "то", "плановое то", "планове то"}


def is_grm_report_request(text: str) -> bool:
    t = str(text or "").lower().strip()
    return t in {"грм", "замена грм", "комплект грм"}


def is_insurance_report_request(text: str) -> bool:
    t = str(text or "").lower().strip()
    return t in {"страховка", "страхування", "страховка?"}


# ===== Telegram handlers =====

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    if ALLOWED_USERS and user_id not in ALLOWED_USERS:
        await update.message.reply_text("⛔ Доступ заборонено")
        return

    text = (update.message.text or "").strip()
    logger.info(f"Incoming message from {user_id}: {text}")
```

```
try:
    if context.user_data.get("waiting_duplicate_confirm"):
        pending_actions = context.user_data.get("pending_actions_after_duplic
        if is_yes_confirm(text):
            context.user_data.pop("waiting_duplicate_confirm", None)
            context.user_data.pop("pending_actions_after_duplicate", None)
            result = write_actions_to_sheet(pending_actions, raw_text=text)
            await update.message.reply_text(result)
            return
        if is_no_confirm(text):
            context.user_data.pop("waiting_duplicate_confirm", None)
            context.user_data.pop("pending_actions_after_duplicate", None)
            await update.message.reply_text("✅ Запис скасовано як дубль.")
            return
        await update.message.reply_text("Напиши «новий» або «дубль».")
        return

    if context.user_data.get("waiting_odometer_anomaly_confirm"):
        pending_actions = context.user_data.get("pending_actions_after_anomal
        if is_yes_confirm(text):
            context.user_data.pop("waiting_odometer_anomaly_confirm", None)
            context.user_data.pop("pending_actions_after_anomaly", None)
            result = write_actions_to_sheet(pending_actions, raw_text=text)
            await update.message.reply_text(result)
            return
        if is_no_confirm(text):
            context.user_data.pop("waiting_odometer_anomaly_confirm", None)
            context.user_data["waiting_odometer_choice_actions"] = True
            context.user_data["pending_actions"] = pending_actions
            context.user_data.pop("pending_actions_after_anomaly", None)
            await update.message.reply_text("Добре. Надішли правильний одомет
            return
        await update.message.reply_text("Напиши «так» для підтвердження або «
        return

    if context.user_data.get("waiting_odometer_choice_actions"):
        pending_actions = context.user_data.get("pending_actions", [])
        numeric_odo = parse_num(text)
        if numeric_odo is not None:
            fill_odometer_for_actions(pending_actions, numeric_odo, estimated
            context.user_data.pop("waiting_odometer_choice_actions", None)
            context.user_data.pop("pending_actions", None)

            first_action = next((a for a in pending_actions if a.get("type")
            if first_action:
                spreadsheet = get_sheet()
```

```
            ws = get_matching_worksheet(spreadsheet, first_action["car_id
            if ws and odometer_is_anomalous(ws, numeric_odo, first_action
                context.user_data["waiting_odometer_anomaly_confirm"] = T
                context.user_data["pending_actions_after_anomaly"] = pend
                await update.message.reply_text("⚠️ Пробіг виглядає нетип
                return

        spreadsheet = get_sheet()
        for action in pending_actions:
            ws = get_matching_worksheet(spreadsheet, action["car_id"])
            if ws and detect_duplicate(ws, action, raw_text=text):
                context.user_data["waiting_duplicate_confirm"] = True
                context.user_data["pending_actions_after_duplicate"] = pe
                await update.message.reply_text("❓ Це новий запис чи дуб
                return

        result = write_actions_to_sheet(pending_actions, raw_text=text)
        await update.message.reply_text(result)
        return

    if is_yes_statistical(text):
        if not pending_actions:
            await update.message.reply_text("Немає даних для обробки.")
            return
        first_action = next((a for a in pending_actions if a.get("type")
        if not first_action:
            result = write_actions_to_sheet(pending_actions, raw_text=tex
            await update.message.reply_text(result)
            return
        estimated = estimate_odometer_for_car(first_action["car_id"], fir
        if not estimated:
            context.user_data.pop("waiting_odometer_choice_actions", None
            context.user_data.pop("pending_actions", None)
            await update.message.reply_text("Не вдалося обчислити середнь
            return
        fill_odometer_for_actions(pending_actions, estimated, estimated=T
        context.user_data.pop("waiting_odometer_choice_actions", None)
        context.user_data.pop("pending_actions", None)

        spreadsheet = get_sheet()
        for action in pending_actions:
            ws = get_matching_worksheet(spreadsheet, action["car_id"])
            if ws and detect_duplicate(ws, action, raw_text=text):
                context.user_data["waiting_duplicate_confirm"] = True
                context.user_data["pending_actions_after_duplicate"] = pe
                await update.message.reply_text("❓ Це новий запис чи дуб
                return
```

```
            result = write_actions_to_sheet(pending_actions, raw_text=text)
            await update.message.reply_text(result)
            return

        await update.message.reply_text("Напиши «так», якщо мені додати серед
        return

    if context.user_data.get("waiting_odometer_choice"):
        pending_data = context.user_data.get("pending_data", {})
        numeric_odo = parse_num(text)
        if numeric_odo is not None:
            pending_data["odometer"] = numeric_odo
            pending_data["odometer_estimated"] = False
            pending_data["missing_fields"] = []
            context.user_data["pending_data"] = pending_data
            context.user_data.pop("waiting_odometer_choice", None)

            spreadsheet = get_sheet()
            ws = get_matching_worksheet(spreadsheet, pending_data["car_id"])
            if ws and odometer_is_anomalous(ws, numeric_odo, pending_data.get
                context.user_data["waiting_odometer_anomaly_confirm"] = True
                context.user_data["pending_actions_after_anomaly"] = [pending
                context.user_data.pop("pending_data", None)
                await update.message.reply_text("⚠️ Пробіг виглядає нетипово
                return
            if ws and detect_duplicate(ws, pending_data, raw_text=text):
                context.user_data["waiting_duplicate_confirm"] = True
                context.user_data["pending_actions_after_duplicate"] = [pendi
                context.user_data.pop("pending_data", None)
                await update.message.reply_text("❓ Це новий запис чи дубль п
                return

            result = write_single_action_to_sheet(pending_data, raw_text=text
            context.user_data.pop("pending_data", None)
            await update.message.reply_text(result)
            return

        if is_yes_statistical(text):
            car_id = pending_data.get("car_id")
            operation_date = pending_data.get("date")
            if not car_id:
                context.user_data.pop("waiting_odometer_choice", None)
                await update.message.reply_text("Спочатку вкажи номер машини.
                return
            estimated = estimate_odometer_for_car(car_id, operation_date)
            if not estimated:
```

```
            context.user_data.pop("waiting_odometer_choice", None)
            await update.message.reply_text("Не вдалося обчислити середнь
            return

        pending_data["odometer"] = estimated
        pending_data["odometer_estimated"] = True
        pending_data["missing_fields"] = []
        context.user_data["pending_data"] = pending_data
        context.user_data.pop("waiting_odometer_choice", None)

        spreadsheet = get_sheet()
        ws = get_matching_worksheet(spreadsheet, pending_data["car_id"])
        if ws and detect_duplicate(ws, pending_data, raw_text=text):
            context.user_data["waiting_duplicate_confirm"] = True
            context.user_data["pending_actions_after_duplicate"] = [pendi
            context.user_data.pop("pending_data", None)
            await update.message.reply_text("❓ Це новий запис чи дубль п
            return

        result = write_single_action_to_sheet(pending_data, raw_text=text
        context.user_data.pop("pending_data", None)
        await update.message.reply_text(result)
        return

    await update.message.reply_text("Напиши «так», якщо мені додати серед
    return

if is_oil_report_request(text):
    report = build_oil_report()
    await update.message.reply_text("🛢️ Стан масла:\n\n" + (report or "Не
    return

if is_grm_report_request(text):
    report = build_grm_report()
    await update.message.reply_text("⚙️ Стан ГРМ:\n\n" + (report or "Нема
    return

if is_insurance_report_request(text):
    report = build_insurance_report()
    await update.message.reply_text("🛡️ Страховка:\n\n" + (report or "Нем
    return

car_id_for_summary = detect_month_summary_request(text)
if car_id_for_summary:
    await update.message.reply_text(monthly_summary(car_id_for_summary))
    return
```

```
# Якщо повідомлення — тільки номер машини (4 цифри) — показуємо картку ав
text_stripped = text.strip()
if re.match(r"^\d{4}$", text_stripped) and text_stripped in KNOWN_CAR_IDS
    car_id = text_stripped
    snapshot = get_data_snapshot()
    rows = next((v for t, v in snapshot.items()
                 if car_id in t or VEHICLE_MAP.get(car_id, "") in t), Non
    lines = [f"🚗 Машина {car_id} ({VEHICLE_MAP.get(car_id, '')})\n"]
    if rows:
        cur_odo = get_current_odometer_from_rows(rows)
        if cur_odo:
            lines.append(f"📍 Поточний одометр: {format_km(cur_odo)} км")
        # Масло
        _, oil_odo = find_last_service(rows, "oil")
        if oil_odo and cur_odo:
            oil_rem = 10000 - (max(cur_odo, oil_odo) - oil_odo)
            oil_icon = get_color_icon(oil_rem, 10000)
            lines.append(f"{oil_icon} Масло: {format_km(oil_rem)} км до р
        # ГРМ
        if car_id not in SKIP_GRM:
            _, grm_odo = find_last_service(rows, "grm")
            if grm_odo and cur_odo:
                grm_rem = 50000 - (max(cur_odo, grm_odo) - grm_odo)
                grm_icon = get_color_icon(grm_rem, 50000)
                lines.append(f"{grm_icon} ГРМ: {format_km(grm_rem)} км до
        # Страховка
        today_d = datetime.now(KYIV_TZ).date()
        best = None
        if len(rows) > 3 and rows[3] and rows[3][0]:
            d, company = parse_insurance_a4(rows[3][0])
            if d:
                best = (d, company)
        if not best:
            for row in rows[7:]:
                if len(row) >= INSURANCE_COMPANY_COL:
                    d = parse_short_date(row[INSURANCE_DATE_COL - 1])
                    company = str(row[INSURANCE_COMPANY_COL - 1]).strip()
                    if d and company:
                        if best is None or d > best[0]:
                            best = (d, company)
        if best:
            days_left = (best[0] - today_d).days
            ins_icon = insurance_days_icon(days_left)
            lines.append(f"{ins_icon} Страховка: {best[0].strftime('%d.%m
    # Місячна статистика
    lines.append("")
    lines.append(monthly_summary(car_id))
```

```
        await update.message.reply_text("\n".join(lines))
        return

    await update.message.reply_text("⏳ Обробляю...")

    heuristic_actions = heuristic_multi_parse(text)
    if heuristic_actions:
        if actions_need_odometer(heuristic_actions):
            context.user_data["pending_actions"] = heuristic_actions
            context.user_data["waiting_odometer_choice_actions"] = True
            await update.message.reply_text("❓ Немає одометра.\nМені додати
            return

        spreadsheet = get_sheet()
        for action in heuristic_actions:
            ws = get_matching_worksheet(spreadsheet, action["car_id"])
            if ws and detect_duplicate(ws, action, raw_text=text):
                context.user_data["waiting_duplicate_confirm"] = True
                context.user_data["pending_actions_after_duplicate"] = heuris
                await update.message.reply_text("❓ Це новий запис чи дубль п
                return

        result = write_actions_to_sheet(heuristic_actions, raw_text=text)
        await update.message.reply_text(result)
        return

    pending_data = context.user_data.get("pending_data")
    parsed = ask_ai(text, existing_data=pending_data)
    if "error" in parsed:
        await update.message.reply_text(f"❌ AI тимчасово недоступний.\n\nДет
        return

    parsed["car_id"] = resolve_car_id(parsed.get("car_id"))
    parsed["date"] = normalize_date_short(parsed.get("date"))
    parsed = apply_special_cases(parsed, text)
    parsed["missing_fields"] = compute_missing_fields(parsed, text)

    if "car_id" in parsed["missing_fields"]:
        context.user_data["pending_data"] = parsed
        await update.message.reply_text(f"❓ Не вдалося визначити машину.\nВк
        return

    if parsed["missing_fields"]:
        context.user_data["pending_data"] = parsed
        if "odometer" in parsed["missing_fields"]:
            context.user_data["waiting_odometer_choice"] = True
            await update.message.reply_text("❓ Немає одометра.\nМені додати
```

```
                return

            await update.message.reply_text(f"❓ Не вистачає даних.\n{ask_for_nex
            return

        spreadsheet = get_sheet()
        ws = get_matching_worksheet(spreadsheet, parsed["car_id"])
        if ws and parsed.get("type") in ["expense", "income"] and parsed.get("odo
            if odometer_is_anomalous(ws, int(parsed["odometer"]), parsed.get("dat
                context.user_data["waiting_odometer_anomaly_confirm"] = True
                context.user_data["pending_actions_after_anomaly"] = [parsed]
                await update.message.reply_text("⚠️ Пробіг виглядає нетипово вели
                return

        if ws and detect_duplicate(ws, parsed, raw_text=text):
            context.user_data["waiting_duplicate_confirm"] = True
            context.user_data["pending_actions_after_duplicate"] = [parsed]
            await update.message.reply_text("❓ Це новий запис чи дубль попереднь
            return

        result = write_single_action_to_sheet(parsed, raw_text=text)
        context.user_data.pop("pending_data", None)
        await update.message.reply_text(result)

    except Exception as e:
        logger.exception("Error")
        await update.message.reply_text(f"❌ Помилка: {str(e)}")


async def handle_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    await update.message.reply_text(
        f"👋 Привіт! Я бот автопарку.\n\n"
        f"Твій Telegram ID: `{user_id}`\n\n"
        f"Я знаю такі машини:\n{', '.join(KNOWN_CAR_IDS)}\n\n"
        f"Команди:\n"
        f"• масло\n"
        f"• грм\n"
        f"• страховка\n"
        f"• 8730 місяць\n"
        f"• ТО 4553\n"
        f"• 8730 приход 3800, долг 200 за дтп, штраф 300 за парковку\n",
        parse_mode="Markdown",
    )


async def handle_cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    for key in [
        "pending_data",
        "pending_actions",
        "waiting_odometer_choice",
        "waiting_odometer_choice_actions",
        "waiting_duplicate_confirm",
        "pending_actions_after_duplicate",
        "waiting_odometer_anomaly_confirm",
        "pending_actions_after_anomaly",
    ]:
        context.user_data.pop(key, None)
    await update.message.reply_text("✅ Поточне введення скасовано.")


def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()
    app.add_handler(CommandHandler("start", handle_start))
    app.add_handler(CommandHandler("cancel", handle_cancel))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_messag
    app.job_queue.run_daily(
        check_service_and_insurance_notifications,
        time=time(9, 15, tzinfo=KYIV_TZ),
        days=(0, 1, 2, 3, 4),  # тільки Пн-Пт
        name="weekday_morning_regulations"
    )
    logger.info("Bot started!")
    app.run_polling(drop_pending_updates=True)


if __name__ == "__main__":
    main()
```